FPGA学习

目前已经完成了乘法器、一维低点数卷积、二维卷积；

正在进行一维卷积点数扩展和FFT。(2021.4.12)

乘法器

for循环简单乘法器

```
module for16bits(
results, x, y
    );
       parameter n= 16;
       output reg[2*n:1]  results;
       input[n:1] x, y;
       integer i;

       always@(x or y)
              begin
                  results= 0;
                  for(i=1; i<=n; i= i+1)
                      if(y[i]) results= results+ (x<<(i-1));
              end
endmodule
```

原码16位乘法器

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
//
// Company:
// Engineer:
//
// Create Date: 2021/01/28 17:53:33
// Design Name:
// Module Name: org4bits
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
```

```
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
//


module org16bits(clk,rst_n,mul_x,mul_y,res
    );
    parameter n= 16;

    input              clk    ;//系统时钟
    input             rst_n   ;//复位信号
    input       [n-1:0] mul_x  ;//被乘数
    input       [n-1:0] mul_y  ;//乘数

    output  reg [2*n-1:0] res    ;//结果

    reg         [n-1:0] mul_p  ;//部分积
    reg                c      ;//进位信号
    reg         [n-1:0] mul_y0 ;//储存乘数
    reg                en     ;//使能信号

    //使能信号模块
    always@(posedge clk or negedge rst_n)begin
        if(rst_n==1'b0)begin
            en<=1'b0;
        end
        else if(mul_x==1'b0||mul_y==1'b0)begin
            en<=1'b0;
        end
        else if(cnt==8'd15)begin
            en<=1'b1;
        end
        else begin
            en<=en;
        end
    end

    //逻辑控制模块(计数)
    reg [7:0] cnt;
    always@(posedge clk or negedge rst_n)begin
        if(rst_n==1'b0)begin
            cnt<=1'b0;
        end
        else if(en==1'b1)begin
            cnt<=8'd0;
        end
        else if(cnt==8'd15)begin
            cnt<=8'd0;
        end
        else begin
            cnt<=cnt+1'b1;
        end
    end

    //逻辑控制模块(加)
    reg con_add_en;
    always@(*)begin
```

```
        if(en==1'b1)begin
            con_add_en=1'b1;
        end
        else if(mul_y0[0]==1'b1)begin
            con_add_en=1'b0;
        end
        else begin
            con_add_en=1'b1;
        end
    end
    //
    always@(posedge clk or negedge rst_n)begin
        if(rst_n==1'b0)begin
            mul_p=1'b0;
            c=1'b0;
            mul_y0=mul_y;
        end
        else if(en==1'b1)begin
            mul_p=mul_p;
            mul_y0=mul_y0;
        end
        else if(con_add_en==1'b0)begin
            {c,mul_p[n-1:0]}=mul_x[n-1:0]+{c,mul_p[n-1:0]};
            {c,mul_p[n-1:0],mul_y0[n-1:0]}={c,mul_p[n-1:0],mul_y0[n-1:0]}>>1;
        end
        else begin
            {c,mul_p[n-1:0],mul_y0[n-1:0]}={c,mul_p[n-1:0],mul_y0[n-1:0]}>>1;
        end
    end

    always@(posedge clk or negedge rst_n)begin//输出控制
        if(rst_n==1'b0)begin
            res<=1'b0;
        end
        else if(mul_x==1'b0||mul_y==1'b0)begin
            res<=1'b0;
        end
        else if(en==1'b1)begin
            res<={mul_p,mul_y0[n-1:0]};
        end
        else begin
            res<=res;
        end
    end

endmodule
```

四位流水线乘法器

```
module ppl_4bits(
mul_a, mul_b, clk, rst_n, result, store0, store1, store2, store3
    );
    input [3:0] mul_a;
    input [3:0] mul_b;
    input clk;
    input rst_n;
```

```
    output [7:0] result;

    output [7:0] store0, store1, store2, store3;
    reg [7:0] store0, store1, store2, store3;
    reg [7:0] add01, add23;
    reg [7:0] result;

    always@(posedge clk or negedge rst_n) begin
        if(!rst_n) begin
            result<= 0;
            store0<= 0;
            store1<= 0;
            store2<= 0;
            store3<= 0;
            add01<= 0;
            add23<= 0;
        end
        else begin
            store0 <= mul_b[0]?{4'b0, mul_a}: 8'b0;
            store1 <= mul_b[1]?{3'b0, mul_a, 1'b0}: 8'b0;
            store2 <= mul_b[2]?{2'b0, mul_a, 2'b0}: 8'b0;
            store3 <= mul_b[3]?{1'b0, mul_a,3'b0}: 8'b0;

          add01<= store0+ store1;
          add23<= store2+ store3;
          result<= add01+ add23;
       end
    end
endmodule
```

卷积运算

四位六点一维卷积

```
module conv1D_v1#(
    parameter CORE3_0 = 2'b01, //conv core
    parameter CORE3_1 = 2'b10,
    parameter CORE3_2 = 2'b11
)
(
    input CLK,
    input RSTn,
    input read,

    output [7:0] CONV_OUT,
    output [3:0] TEST_OUT_0,
    output [3:0] TEST_OUT_1
);

reg [3:0] ROM [5:0];                    //memory
reg [2:0] rd;
reg [1:0] rd1;
```

```
reg [1:0] rd2;
wire [2:0] addr;                    //memory address
reg [7:0] CONV_OUT_REG;                 //regsiter of conv_out
reg [3:0] WIN0;                            //windows data
reg [3:0] WIN1;
reg [3:0] WIN2;
integer i=0;

//initial                                   //设置ROM里面的数据（相当于卷积对象）
//begin
//  ROM[ 0] = 4'b0001;//1
//  ROM[ 1] = 4'b0010;
//  ROM[ 2] = 4'b0011;
//  ROM[ 3] = 4'b0100;
//  ROM[ 4] = 4'b0101;
//  ROM[ 5] = 4'b0110;//6
//end

initial
begin
    ROM[ 0] = 4'b0001;//1
    ROM[ 1] = 4'b0010;
    ROM[ 2] = 4'b0011;
    ROM[ 3] = 4'b0100;
    ROM[ 4] = 4'b0101;
    ROM[ 5] = 4'b0110;//6

    begin
        for(i=0;i<6;i=i+1)
            begin
                ROM[i]=4'b0001;
            end
    end
end



always @ (posedge CLK or negedge RSTn)//用win来装等长于卷积核的计算数据
begin
    if (!RSTn)
        begin
        WIN0 <= 2'bx;
        WIN1 <= 2'bx;
        WIN2 <= 2'bx;
        rd <= 0;
        end
    else if (read)
        begin
            WIN0 <= ROM[addr];
            WIN1 <= ROM[addr + 2'b01];
            WIN2 <= ROM[addr + 2'b10];

            if(rd== 3) rd<= 0;
            else
             rd <= rd + 1'b1;
        end
end
```

```
always @ ( posedge CLK or negedge RSTn )
begin
    if (!RSTn)
        CONV_OUT_REG <= 0;
    else
        CONV_OUT_REG <= WIN0 * CORE3_2 + WIN1 * CORE3_1 + WIN2 * CORE3_0;
//      CONV_OUT_REG <= WIN0 * CORE3_2;
//      CONV_OUT_REG <= WIN1 * CORE3_1;
//      CONV_OUT_REG <= WIN2 * CORE3_0;
end


assign CONV_OUT = CONV_OUT_REG;
assign addr = rd; //在rd前面放一个中转的addr
assign TEST_OUT_0= WIN1;
assign TEST_OUT_1= addr + 2'b01;
endmodule
```

二维卷积器

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
//
// Company:
// Engineer:
//
// Create Date: 2021/03/20 14:35:34
// Design Name:
// Module Name: my_conv_2
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
//


module my_conv_2#(
    parameter convcell0=2'b10,
    parameter convcell1=2'b01,
    parameter convcell2=2'b11,
    parameter convcell3=2'b10,
    parameter convcell4=2'b01,
    parameter convcell5=2'b10,
    parameter convcell6=2'b00,
    parameter convcell7=2'b11,
    parameter convcell8=2'b01
)
(
```

```
    input clk,
    input read,
    input rst,
    output[7:0] conv_out
    );
    reg[3:0] now[35:0];
    reg[3:0] coning[8:0];
    reg[6:0] loc;
    wire[7:0] conv_out;
    reg[7:0] outtemp;
    integer i;

    initial
    begin
        now[0]=1'h7;
        now[1]=1'h5;
        now[2]=1'h8;
        now[3]=1'ha;
        now[4]=1'hb;
        now[5]=1'h3;
        now[6]=1'h6;
        now[7]=1'h2;
        now[8]=1'h7;
        now[9]=1'hb;
        now[10]=1'hc;
        now[11]=1'h9;
        now[12]=1'hd;
        now[13]=1'he;
        now[14]=1'hf;
        now[15]=1'h3;
        now[16]=1'h7;
        now[17]=1'h8;
        now[18]=1'h3;
        now[19]=1'h5;
        now[20]=1'h8;
        now[21]=1'h3;
        now[22]=1'h1;
        now[23]=1'h0;
        now[24]=1'h7;
        now[25]=1'h4;
        now[26]=1'ha;
        now[27]=1'hb;
        now[28]=1'hc;
        now[29]=1'ha;
        now[30]=1'h5;
        now[31]=1'hb;
        now[32]=1'hd;
        now[33]=1'hf;
        now[34]=1'h5;
        now[35]=1'h2;
    end

    always@(posedge clk or posedge rst)
    begin
    if(rst)
    begin
        for(i=0;i<36;i=i+1)begin
            coning[i]<=0;//将当前读取数据取零
```

```
        end
    end
    else if(read)
    begin
        coning[0]<=now[loc];
        coning[1]<=now[loc+4'b0001];
        coning[2]<=now[loc+4'b0010];
        coning[3]<=now[loc+4'b0110];
        coning[4]<=now[loc+4'b0111];
        coning[5]<=now[loc+4'b1000];
        coning[6]<=now[loc+4'b1100];
        coning[7]<=now[loc+4'b1101];
        coning[8]<=now[loc+4'b1110];
    end
    end

    always@(posedge clk or posedge rst)
    begin
        if(rst)
        begin
            loc<=0;//如果rst，则重新开始，将位置指针置0
        end
        else
        begin
            loc<=loc+2'b1;
            case(loc)
            6'd3:loc<=loc+2'b11;
            6'd9:loc<=loc+2'b11;
            6'd15:loc<=loc+2'b11;
            6'd21:loc<=0;
            endcase
        end
    end

    always@(posedge clk or posedge rst)
    begin
        if(rst)
        begin
            outtemp<=0;
        end
        else
        begin

 outtemp<=coning[0]*convcell0+coning[1]*convcell1+coning[2]*convcell2+coning[3]*
convcell3+coning[4]*convcell4+coning[5]*convcell5+coning[6]*convcell6+coning[7]*
convcell7+coning[8]*convcell8;
        end
    end

    assign conv_out=outtemp;
endmodule
```

暂未完成的代码

256点一维卷积器

```
module conv1D_256#(
    parameter CORE3_0 = 2'b01, //卷积核1、2、3
    parameter CORE3_1 = 2'b01,
    parameter CORE3_2 = 2'b01
)
(
    input CLK,
    input RSTn,
    input read,

    output [7:0] CONV_OUT,
    output [3:0] TEST_OUT_0,
    output [3:0] TEST_OUT_1
);

reg [3:0] ROM [255:0];                    //memory
reg [8:0] rd;
wire [8:0] addr;                   //memory address
reg [7:0] CONV_OUT_REG;                 //regsiter of conv_out
reg [3:0] WIN0;                           //windows data
reg [3:0] WIN1;
reg [3:0] WIN2;
integer i=0;



initial//for循环设置参数
begin
    for(i=0;i<256;i=i+1)
        begin
            ROM[i]=i;
        end
end

always @ (posedge CLK or negedge RSTn)//用win来装等长于卷积核的计算数据
begin
    if (!RSTn)
        begin
        WIN0 <= 2'bx;
        WIN1 <= 2'bx;
        WIN2 <= 2'bx;
        rd <= 0;
        end
    else if (read)
        begin
            WIN0 <= ROM[addr];
            WIN1 <= ROM[addr + 2'b01];
            WIN2 <= ROM[addr + 2'b10];

            if(rd== 3) rd<= 0;
            else
             rd <= rd + 1'b1;
```

```
        end
end

always @ ( posedge CLK or negedge RSTn )
begin
    if (!RSTn)
        CONV_OUT_REG <= 0;
    else
        CONV_OUT_REG <= WIN0 * CORE3_2 + WIN1 * CORE3_1 + WIN2 * CORE3_0;
end


assign CONV_OUT = CONV_OUT_REG;
assign addr = rd; //在rd前面放一个中转的addr
assign TEST_OUT_0= WIN1;
assign TEST_OUT_1= addr + 2'b01;
endmodule
```

FFT

1.for8（top）

```
`timescale 1ns / 1ps

module fft8 (
     input                      clk,
     input                      rstn,
     input                      en,

     input signed [23:0]        x0_real,
     input signed [23:0]        x0_imag,
     input signed [23:0]        x1_real,
     input signed [23:0]        x1_imag,
     input signed [23:0]        x2_real,
     input signed [23:0]        x2_imag,
     input signed [23:0]        x3_real,
     input signed [23:0]        x3_imag,
     input signed [23:0]        x4_real,
     input signed [23:0]        x4_imag,
     input signed [23:0]        x5_real,
     input signed [23:0]        x5_imag,
     input signed [23:0]        x6_real,
     input signed [23:0]        x6_imag,
     input signed [23:0]        x7_real,
     input signed [23:0]        x7_imag,

     output                     valid,
     output signed [23:0]       y0_real,
     output signed [23:0]       y0_imag,
     output signed [23:0]       y1_real,
     output signed [23:0]       y1_imag,
     output signed [23:0]       y2_real,
     output signed [23:0]       y2_imag,
     output signed [23:0]       y3_real,
```

```
    output signed [23:0]     y3_imag,
    output signed [23:0]     y4_real,
    output signed [23:0]     y4_imag,
    output signed [23:0]     y5_real,
    output signed [23:0]     y5_imag,
    output signed [23:0]     y6_real,
    output signed [23:0]     y6_imag,
    output signed [23:0]     y7_real,
    output signed [23:0]     y7_imag
    );

    //operating data
    wire signed [23:0]             xm_real [3:0] [7:0];
    wire signed [23:0]             xm_imag [3:0] [7:0];
    wire                           en_connect [15:0] ;
    assign                         en_connect[0] = en;
    assign                         en_connect[1] = en;
    assign                         en_connect[2] = en;
    assign                         en_connect[3] = en;

    wire signed [15:0]             factor_real [3:0] ;
    wire signed [15:0]             factor_imag [3:0];
    assign factor_real[0]        = 16'h2000; //1
    assign factor_imag[0]        = 16'h0000; //0
    assign factor_real[1]        = 16'h16a0; //sqrt(2)/2
    assign factor_imag[1]        = 16'he95f; //-sqrt(2)/2
    assign factor_real[2]        = 16'h0000; //0
    assign factor_imag[2]        = 16'he000; //-1
    assign factor_real[3]        = 16'he95f; //-sqrt(2)/2
    assign factor_imag[3]        = 16'he95f; //-sqrt(2)/2

    //输入初始化，和码位有关倒置
    assign xm_real[0][0] = x0_real;
    assign xm_real[0][1] = x4_real;
    assign xm_real[0][2] = x2_real;
    assign xm_real[0][3] = x6_real;
    assign xm_real[0][4] = x1_real;
    assign xm_real[0][5] = x5_real;
    assign xm_real[0][6] = x3_real;
    assign xm_real[0][7] = x7_real;
    assign xm_imag[0][0] = x0_imag;
    assign xm_imag[0][1] = x4_imag;
    assign xm_imag[0][2] = x2_imag;
    assign xm_imag[0][3] = x6_imag;
    assign xm_imag[0][4] = x1_imag;
    assign xm_imag[0][5] = x5_imag;
    assign xm_imag[0][6] = x3_imag;
    assign xm_imag[0][7] = x7_imag;

    //butter instantiaiton
    //integer              index[11:0] ;
    genvar               m, k;
    generate
    //3 stage
    for(m=0; m<=2; m=m+1) begin: stage
        for (k=0; k<=3; k=k+1) begin: unit

            butterfly           u_butter
```

```
            (
                .clk        (clk) ,
                .rstn       (rstn) ,
                .en         (en_connect[m*4 + k]) ,
                        //是否再组内？组编号+组内编号：下组编号+新组内编号
                .xp_real    (xm_real[ m ] [k[m:0] < (1<<m) ?
                            k[3:m] + k[m:0] :
                            (k[3:m] << (m+1)) + (k[m:0]-(1<<m))] ),
                .xp_imag    (xm_imag[ m ] [k[m:0] < (1<<m) ?
                            k[3:m] + k[m:0] :
                            (k[3:m] << (m+1)) + (k[m:0]-(1<<m))] ),
                .xq_real    (xm_real[ m ] [(k[m:0] < (1<<m) ?
                            k[3:m] + k[m:0] :
                            (k[3:m] << (m+1)) + (k[m:0]-(1<<m))) + (1<<m) ]),

                             //增加蝶形单元两个输入端口间距离
                .xq_imag    (xm_imag[ m ] [(k[m:0] < (1<<m) ?
                            k[3:m] + k[m:0] :
                            (k[3:m] << (m+1)) + (k[m:0]-(1<<m))) + (1<<m) ]),

                .factor_real(factor_real[k[m:0]<(1<<m)?
                             k[m:0] : k[m:0]-(1<<m) ]),

                .factor_imag(factor_imag[k[m:0]<(1<<m)?
                             k[m:0] : k[m:0]-(1<<m) ]),

                //output data
                .valid      (en_connect[ (m+1)*4 + k ]  ),

                .yp_real    (xm_real[ m+1 ][k[m:0] < (1<<m) ?
                            (k[3:m] << (m+1)) + k[m:0] :
                            (k[3:m] << (m+1)) + (k[m:0]-(1<<m))] ),

                .yp_imag    (xm_imag[ m+1 ][(k[m:0]) < (1<<m) ?
                            (k[3:m] << (m+1)) + k[m:0] :
                            (k[3:m] << (m+1)) + (k[m:0]-(1<<m))] ),

                .yq_real    (xm_real[ m+1 ][(k[m:0] < (1<<m) ?
                            (k[3:m] << (m+1)) + k[m:0] :
                            (k[3:m] << (m+1)) + (k[m:0]-(1<<m))) + (1<<m) ]),

                .yq_imag    (xm_imag[ m+1 ][((k[m:0]) < (1<<m) ?
                            (k[3:m] << (m+1)) + k[m:0] :
                            (k[3:m] << (m+1)) + (k[m:0]-(1<<m))) + (1<<m) ])
            );
        end
    end
endgenerate

assign      valid = en_connect[12];
assign      y0_real = xm_real[3][0] ;
assign      y0_imag = xm_imag[3][0] ;
assign      y1_real = xm_real[3][1] ;
assign      y1_imag = xm_imag[3][1] ;
assign      y2_real = xm_real[3][2] ;
assign      y2_imag = xm_imag[3][2] ;
assign      y3_real = xm_real[3][3] ;
assign      y3_imag = xm_imag[3][3] ;
```

```
    assign     y4_real = xm_real[3][4] ;
    assign     y4_imag = xm_imag[3][4] ;
    assign     y5_real = xm_real[3][5] ;
    assign     y5_imag = xm_imag[3][5] ;
    assign     y6_real = xm_real[3][6] ;
    assign     y6_imag = xm_imag[3][6] ;
    assign     y7_real = xm_real[3][7] ;
    assign     y7_imag = xm_imag[3][7] ;

endmodule
```

2.蝴蝶变换

```
`timescale 1ns / 1ps

module butterfly
    (
     input                          clk,
     input                          rstn,
     input                          en,
     input signed [23:0]            xp_real, // Xm_p
     input signed [23:0]            xp_imag,
     input signed [23:0]            xq_real, // Xm_q
     input signed [23:0]            xq_imag,
     input signed [15:0]            factor_real, // Wnr
     input signed [15:0]            factor_imag,

     output                         valid,
     output signed [23:0]           yp_real, //Xm+1_p
     output signed [23:0]           yp_imag,
     output signed [23:0]           yq_real, //Xm+1_q
     output signed [23:0]           yq_imag
     );

    reg [4:0]                       en_r ;
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            en_r   <= 'b0 ;
        end
        else begin
            en_r   <= {en_r[3:0], en} ;
        end
    end

    //Xm_q mutiply and Xm_p delay
    reg signed [39:0] xq_wnr_real0;
    reg signed [39:0] xq_wnr_real1;
    reg signed [39:0] xq_wnr_imag0;
    reg signed [39:0] xq_wnr_imag1;
    reg signed [39:0] xp_real_d;
    reg signed [39:0] xp_imag_d;
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            xp_real_d    <= 'b0;
            xp_imag_d    <= 'b0;
            xq_wnr_real0 <= 'b0;
            xq_wnr_real1 <= 'b0;
```

```
            xq_wnr_imag0 <= 'b0;
            xq_wnr_imag1 <= 'b0;
        end
        else if (en) begin
            xq_wnr_real0 <= xq_real * factor_real;
            xq_wnr_real1 <= xq_imag * factor_imag;
            xq_wnr_imag0 <= xq_real * factor_imag;
            xq_wnr_imag1 <= xq_imag * factor_real;
            xp_real_d    <= {{4{xp_real[23]}}, xp_real[22:0], 13'b0};
            xp_imag_d    <= {{4{xp_imag[23]}}, xp_imag[22:0], 13'b0};
        end
    end

    // get Xm_q mutiplied-results and Xm_p delay again
    reg signed [39:0] xp_real_d1;
    reg signed [39:0] xp_imag_d1;
    reg signed [39:0] xq_wnr_real;
    reg signed [39:0] xq_wnr_imag;
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            xp_real_d1     <= 'b0;
            xp_imag_d1     <= 'b0;
            xq_wnr_real    <= 'b0 ;
            xq_wnr_imag    <= 'b0 ;
        end
        else if (en_r[0]) begin
            xp_real_d1     <= xp_real_d;
            xp_imag_d1     <= xp_imag_d;
            //设置位宽余量，防止数据溢出
           xq_wnr_real    <= xq_wnr_real0 - xq_wnr_real1 ;
            xq_wnr_imag    <= xq_wnr_imag0 + xq_wnr_imag1 ;
      end
    end

   //results
    reg signed [39:0] yp_real_r;
    reg signed [39:0] yp_imag_r;
    reg signed [39:0] yq_real_r;
    reg signed [39:0] yq_imag_r;
    always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            yp_real_r      <= 'b0;
            yp_imag_r      <= 'b0;
            yq_real_r      <= 'b0;
            yq_imag_r      <= 'b0;
        end
        else if (en_r[1]) begin
            yp_real_r      <= xp_real_d1 + xq_wnr_real;
            yp_imag_r      <= xp_imag_d1 + xq_wnr_imag;
            yq_real_r      <= xp_real_d1 - xq_wnr_real;
            yq_imag_r      <= xp_imag_d1 - xq_wnr_imag;
        end
    end

    //discard the low 13bits because of Wnr
    assign yp_real = {yp_real_r[39], yp_real_r[13+23:13]};
    assign yp_imag = {yp_imag_r[39], yp_imag_r[13+23:13]};
    assign yq_real = {yq_real_r[39], yq_real_r[13+23:13]};
```

```
    assign yq_imag = {yq_imag_r[39], yq_imag_r[13+23:13]};
    assign valid   = en_r[2];

endmodule
```